UNIVERSITY INSTITUTE OF COMPUTING

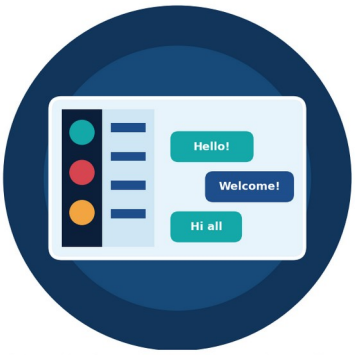
Bachelor of Computer Applications (BCA)

PROJECT BASED LEARNING (PBL) / CASE STUDY REPORT

on

DISCORD – ONLINE CHAT AND COMMUNICATION PLATFORM

An Educational Software Engineering Case Study



Schematic of a chat and communication platform

Subject: Software Engineering

Name	Gursimran Singh
UID	25BCD10268
Course	BCA
Semester	3rd
Submitted To	Jitendra Kumar
Academic Session	2026–2027

University Institute of Computing

Note: This is an independent educational case study. It is not affiliated with or endorsed by Discord Inc.

CERTIFICATE

This is to certify that the PBL / Case Study report entitled “Discord – Online Chat and Communication Platform” is a bonafide work carried out by Gursimran Singh, UID 25BCD10268, a student of BCA, 3rd Semester, University Institute of Computing, in partial fulfilment of the requirements of the subject Software Engineering during the Academic Session 2026–2027.

The work has been completed under my guidance and, to the best of my knowledge, has not been submitted elsewhere for any other purpose. This report is an educational software engineering case study.

_____

Jitendra Kumar
Subject Teacher / Project Guide

Date: _____ Place: _____

DECLARATION

I, Gursimran Singh, UID 25BCD10268, a student of BCA, 3rd Semester, University Institute of Computing, hereby declare that this PBL / Case Study report entitled “Discord – Online Chat and Communication Platform” is my own work, prepared as part of the subject Software Engineering for the Academic Session 2026–2027.

The information in this report has been collected from textbooks and reliable sources listed in the References section. I have not copied any report or project of another student. I further declare that this is an educational case study; the design in this report is my own simplified, conceptual design and does not describe the real internal systems of any company.

Date: _____

Place: _____

Signature of Student
Gursimran Singh (UID: 25BCD10268)

ACKNOWLEDGMENT

I would like to express my sincere gratitude to my subject teacher and guide, Mr. Jitendra Kumar, for his valuable guidance, encouragement and support throughout this case study. His explanations of Software Engineering concepts helped me understand how real systems are planned, designed and tested.

I am thankful to the University Institute of Computing for giving me the opportunity to work on a Project Based Learning (PBL) topic. I also thank all faculty members for creating a good learning environment.

Finally, I thank my family and friends for their constant support and motivation during the preparation of this report.

Gursimran Singh
BCA, 3rd Semester

ABSTRACT

Online chat and communication platforms such as Discord let people form communities, talk in text and voice channels, share files and manage who can do what. Behind a simple chat screen there is a large software system that must deliver messages in real time, protect user accounts and handle misuse. This report studies a Discord-style chat and communication platform as an educational Software Engineering case study.

The report explains the problem statement, objectives and scope, compares older ways of communicating with the proposed platform, and describes the system components. It documents the software requirements and uses standard software engineering models: a layered architecture, a flowchart, a UML use case diagram, Data Flow Diagrams (DFD), an Entity-Relationship (ER) diagram, and UML class, sequence and state diagrams. The Software Development Life Cycle (SDLC), implementation overview, security and risk management, sample test cases and ethical considerations are also discussed, followed by advantages, limitations and future scope.

This work is purely educational. The design is a simplified, conceptual design made by the student; it does not describe the real internal systems of Discord and uses no real user data.

Keywords: *Chat platform, real-time messaging, roles and permissions, SDLC, UML, DFD, ER diagram, software testing, security.*

TABLE OF CONTENTS

Certificate....i
Declaration....ii
Acknowledgment....iii
Abstract....iv
Table of Contents....v
List of Figures....vi
List of Tables....vii
Abbreviations and Glossary....viii
Chapter 1: Introduction....1
Chapter 2: Problem Statement....2
Chapter 3: Objectives....2
Chapter 4: Scope of the Project....3
Chapter 5: Existing System....3
Chapter 6: Proposed System....4
Chapter 7: System Components....5
Chapter 8: Software Requirements....5
Chapter 9: System Architecture....6
Chapter 10: Working of the System....7
Chapter 11: System Flowchart....8
Chapter 12: UML Use Case Diagram....9
Chapter 13: Data Flow Diagram (DFD)....10
Chapter 14: Database Design....11
Chapter 15: UML Design Models....11
Chapter 16: Software Development Life Cycle (SDLC)....13
Chapter 17: Implementation Overview....14
Chapter 18: Security and Risk Management....15
Chapter 19: Software Testing....15
Chapter 20: Ethical and Professional Considerations....16
Chapter 21: Advantages....17
Chapter 22: Limitations....17
Chapter 23: Future Scope....17
Chapter 24: Conclusion....18
References....19
Appendix A: Requirements Traceability Matrix....20

LIST OF FIGURES

Figure 1.1: Simplified schematic of a chat and communication platform....1
Figure 6.1: Conceptual block diagram of the proposed system....4
Figure 7.1: Main components of the chat platform....5
Figure 9.1: Three-layer architecture of the chat platform....6
Figure 11.1: Flowchart of sending a message....8
Figure 12.1: UML use case diagram....9
Figure 13.1: Level 0 DFD....10
Figure 13.2: Level 1 DFD....10
Figure 14.1: Entity-Relationship (ER) diagram....11
Figure 15.1: UML class diagram of the main classes....12
Figure 15.2: UML sequence diagram for sending a message....12
Figure 15.3: UML state diagram of user presence....13
Figure 16.1: SDLC phases....13

LIST OF TABLES

Table 2.1: Main challenges..... 2
Table 6.1: Existing versus proposed system..... 4
Table 7.1: Components and their purpose..... 5
Table 8.1: Functional requirements..... 5
Table 8.2: Non-functional requirements..... 6
Table 9.1: Software modules and responsibilities..... 6
Table 10.1: Examples of system behaviour..... 7
Table 12.1: Actors and use cases..... 9
Table 12.2: Use case description: Send Message..... 9
Table 14.1: Table structure..... 11
Table 16.1: SDLC phases and activities..... 13
Table 18.1: Security measures..... 15
Table 18.2: Sample risk table..... 15
Table 19.1: Sample test cases..... 15
Table A.1: Requirement traceability..... 20

ABBREVIATIONS AND GLOSSARY

Abbreviation	Full Form
BCA	Bachelor of Computer Applications
PBL	Project Based Learning
SDLC	Software Development Life Cycle
UML	Unified Modeling Language
DFD	Data Flow Diagram
ER	Entity-Relationship
API	Application Programming Interface
TLS	Transport Layer Security
PK / FK	Primary Key / Foreign Key
FR / NFR	Functional / Non-Functional Requirement

Term	Meaning
Server	A community space where members chat.
Channel	A text or voice room inside a server.
Role	A set of permissions given to members.
Presence	A user’s status, such as online or idle.
Real-time	Delivered almost instantly.
Moderation	Controlling content and behaviour in a community.

Chapter 1: Introduction

1.1 Background

People now use the internet to study, play, work and meet friends. Chat and communication platforms make this easy by putting text chat, voice chat, file sharing and community tools in one place. Discord is a well-known example of such a platform. This case study uses a Discord-style platform to learn how a real-time communication system is designed.

1.2 Basic Concepts

- Server: a community space created by a user, where members chat.
- Channel: a text or voice room inside a server, usually for one topic.
- Member and role: a member belongs to a server; roles decide what each member may do.
- Direct message: a private chat between users.
- Presence: the status of a user, such as online, idle or offline.

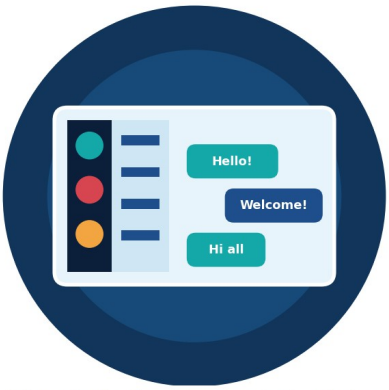


Figure 1.1: Simplified schematic of a chat and communication platform

1.3 Why is it a Good Software Engineering Case Study?

A chat platform looks simple but contains many important ideas: several types of users, real-time delivery, permissions, databases, security and moderation. These make it a good example for learning requirements, design models and the SDLC.

1.4 About this Case Study

This report applies Software Engineering concepts to a Discord-style platform. It uses publicly known features only as examples. The architecture and models shown here are the student's own simplified, conceptual design; they do not describe how Discord is actually built. It is an independent educational study and uses no real user data.

1.5 Organisation of the Report

Chapters 2 to 4 define the problem, objectives and scope. Chapters 5 to 8 describe the existing and proposed systems, components and requirements. Chapters 9 to 15 present the design models. Chapters 16 to 20 cover SDLC, implementation, security, testing and ethics. Chapters 21 to 24 give advantages, limitations, future scope and the conclusion.

Chapter 2: Problem Statement

Groups of people often communicate using many separate tools such as email, SMS and phone calls. Messages get scattered, old information is hard to find, and there is no easy way to organise topics or control who can say or do what. Large online groups also face spam and harmful behaviour.

2.1 Problems with Scattered Communication

- Messages are spread across different apps and inboxes.
- There are no topic-wise rooms, so conversations get mixed up.
- It is hard to find older messages or shared files.
- There is no role system to control who can post, delete or invite.
- Moderating spam or abuse is difficult.
- Delays or missed messages when delivery is not real time.

2.2 Problem Definition

The problem is to design software for a chat and communication platform that lets users register and log in, join or create servers and channels, send and receive messages in real time, manage roles and permissions, moderate content and receive notifications, while keeping accounts and data secure.

2.3 Key Challenges for Software Engineers

Table 2.1: Main challenges

Challenge	Meaning
Real-time delivery	Messages must reach all members quickly.
Scalability	The system must handle many users and messages.
Security	Accounts and private data must be protected.
Moderation	Spam and harmful content must be controlled.
Usability	Screens must be simple and quick to use.

Chapter 3: Objectives

The main objectives of this case study are:

- To understand how an online chat and communication platform works.
- To identify the functional and non-functional requirements of the system.
- To design the system using an architecture diagram, flowchart, UML diagrams, DFD and ER diagram.
- To apply SDLC phases and basic software testing ideas.
- To study security, moderation and risk management.
- To understand ethical responsibility in online communication software.
- To present the work in a clear report.

Chapter 4: Scope of the Project

Scope defines what this case study covers and what it does not cover.

4.1 Included

- Requirements analysis (functional and non-functional).
- Software architecture and database design.
- Flowchart, UML diagrams, DFD and ER diagram.
- SDLC explanation, pseudocode and sample test cases.
- Security, moderation and risk discussion.

4.2 Excluded

- Building the complete working software.
- The real internal design or source code of Discord.
- Real user data or real servers.
- Detailed audio and video streaming technology.
- Payment features, bots and third-party integrations.

4.3 Assumptions

- Users have an internet connection and a device with a browser or app.
- Every user, server, channel and message has a unique ID.
- Voice features are treated at a high level only.

Chapter 5: Existing System

Before dedicated chat platforms, groups used email, SMS, phone calls and basic group chats. Each tool has a different purpose, and none of them gives organised rooms, roles and community tools in one place.

5.1 Limitations of the Existing System

- Conversations are scattered across several tools.
- No separate channels for different topics.
- No roles or permissions for group members.
- Limited moderation tools against spam.
- Hard to search old messages and shared files.
- Email is slow for quick conversation.

5.2 Software Engineering View

From a software point of view, the existing tools are not designed as one organised community system. This shows why a purpose-built chat platform is a good case study.

Chapter 6: Proposed System

The proposed system is a Discord-style chat platform. Users log in through the access control module, and the main services handle servers, channels, messages, roles and presence. Data is stored in a central database. Separate modules handle moderation and notifications.

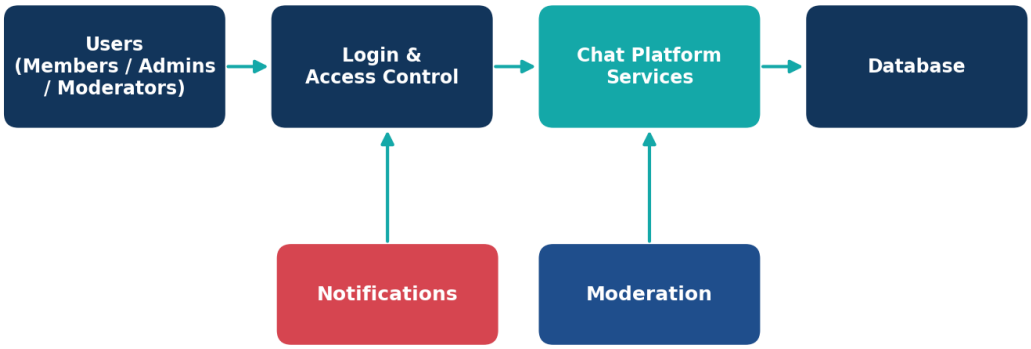


Figure 6.1: Conceptual block diagram of the proposed system

6.1 Key Features

- Registration and login with secure passwords.
- Create and join servers using invites.
- Text and voice channels inside servers.

- Real-time message delivery.
- Roles and permissions for members.
- Moderation tools: delete messages, mute or ban members.
- Presence status and notifications.

Table 6.1: Existing versus proposed system

Point	Existing System	Proposed System
Communication	Many separate tools	One platform
Topics	Mixed together	Separate channels
Control	No roles	Roles and permissions
Moderation	Limited	Built-in tools
Delivery	Often delayed	Real time

Chapter 7: System Components

The platform needs several hardware and software components that work together.

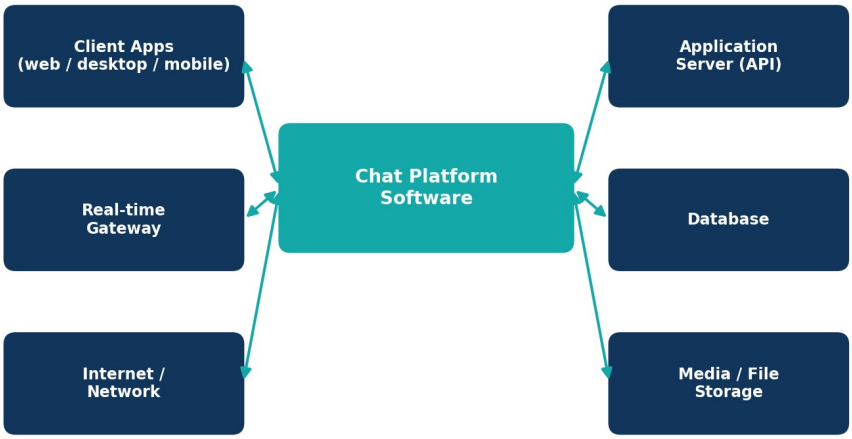


Figure 7.1: Main components of the chat platform

Table 7.1: Components and their purpose

Component	Purpose
Client apps	Web, desktop or mobile apps that users use to chat.
Real-time gateway	Keeps connections open so messages are delivered instantly.
Application server (API)	Runs the business rules such as login, permissions and messages.
Database	Stores users, servers, channels and messages.
Media / file storage	Stores images and files shared in chats.
Internet / network	Connects the clients to the servers.

Chapter 8: Software Requirements

Requirements describe what the system must do (functional) and how well it must do it (non-functional).

Table 8.1: Functional requirements

ID	Requirement
FR1	The system shall allow users to register and log in.
FR2	The system shall allow users to create servers and join them with an invite.
FR3	The system shall allow admins to create text and voice channels.
FR4	The system shall allow members to send and receive messages in real time.
FR5	The system shall allow admins to assign roles and permissions.
FR6	The system shall allow moderators to delete messages and mute or ban members.
FR7	The system shall show presence status and send notifications.
FR8	The system shall allow users to edit their profile and settings.

Table 8.2: Non-functional requirements

Quality	Description
Performance	Messages should be delivered quickly.
Scalability	The system should handle growth in users and messages.
Security	Passwords protected; data sent over secure connections.
Availability	The service should be available most of the time.
Usability	Screens should be simple and easy to learn.
Maintainability	The design should be easy to update and fix.

8.3 Constraints

- Users must be identified by unique IDs and verified by login.
- Permissions must always be checked before an action is allowed.
- The system depends on the user’s internet connection.

Chapter 9: System Architecture

A three-layer architecture is used: client, application service and data. Each layer talks only to the layer next to it, which keeps the design simple and easier to test.

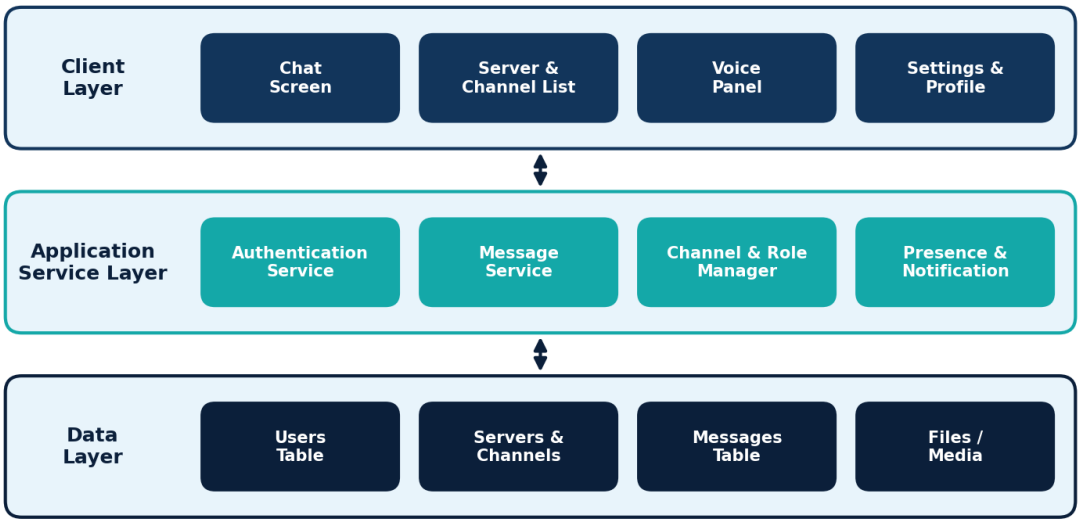


Figure 9.1: Three-layer architecture of the chat platform

Table 9.1: Software modules and responsibilities

Module	Responsibility
Authentication Service	Register users, check passwords and manage login.
Message Service	Receive, save and deliver messages.
Channel & Role Manager	Manage servers, channels, roles and permissions.
Presence & Notification Service	Track online status and send notifications.
Moderation Module	Delete messages and mute or ban members.
Chat Screen / Client UI	Show channels and messages to the user.
Database Tables	Store users, servers, channels, messages and files.

Chapter 10: Working of the System

The workflow of the system is explained step by step below.

1. The user registers or logs in; the system checks the details.
2. The user joins a server using an invite or creates a new server.
3. The user selects a channel inside the server.
4. The user types a message and presses send.
5. The system checks that the user has permission to post in that channel.

6. The message is validated, saved in the database and delivered to online members.
7. Moderators can delete messages or mute and ban members if rules are broken.
8. Presence status and notifications are updated continuously.

10.1 Normal and Exceptional Situations

Table 10.1: Examples of system behaviour

Situation	System Response
Wrong password	Access denied; error message shown.
No permission for channel	Message refused; “access denied” shown.
Empty or too long message	Message rejected with an error.
Member breaks rules	Moderator deletes message or mutes / bans the member.

Chapter 11: System Flowchart

The flowchart shows the process of sending a message. The oval is the start or end, rectangles are process steps, diamonds are decisions, and the red boxes are the refusal paths.

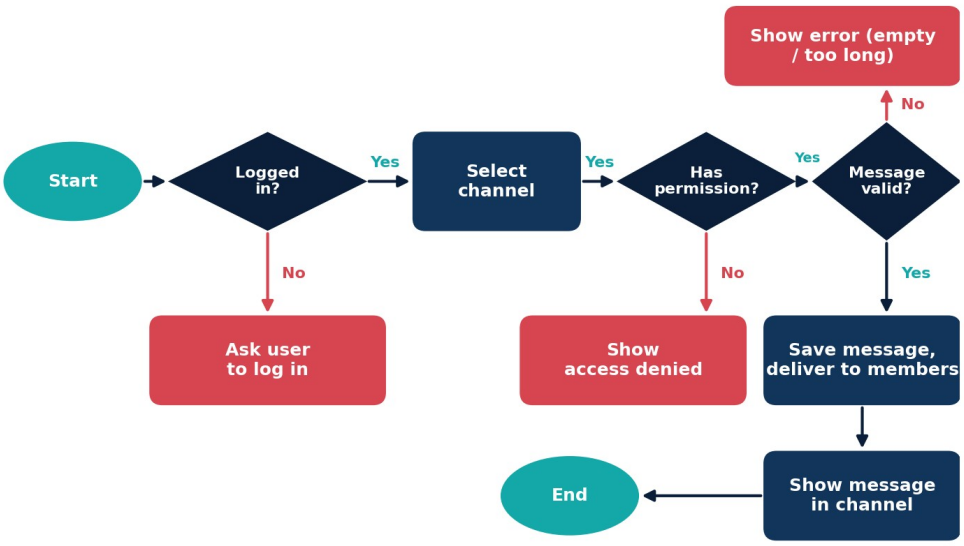


Figure 11.1: Flowchart of sending a message

Explanation: After start, the system checks that the user is logged in. The user selects a channel, and the system checks permission. The message is then validated. If every check passes, the message is saved, delivered to members and shown in the channel. Otherwise an error or access denied message is shown.

Chapter 12: UML Use Case Diagram

A use case diagram shows the actors (users) and how they interact with the system.

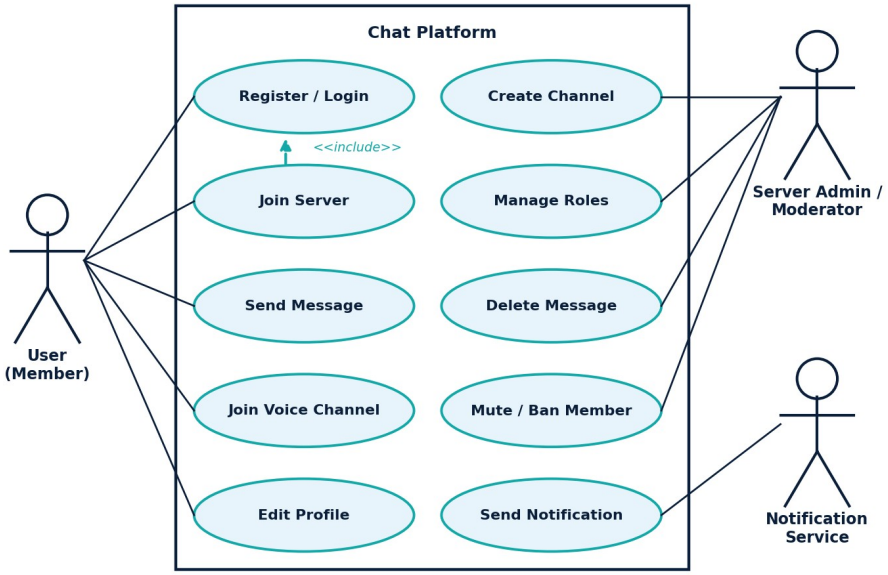


Figure 12.1: UML use case diagram

Table 12.1: Actors and use cases

Actor	Use Cases
User (Member)	Register / login, join server, send message, join voice channel, edit profile.
Server Admin / Moderator	Create channel, manage roles, delete message, mute / ban member.
Notification Service	Send notification (external or system actor).

The dashed line shows that “Join Server” includes “Register / Login”: a user must log in first.

Table 12.2: Use case description: Send Message

Item	Description
Actor	User (Member)
Precondition	User is logged in and is a member of the server.
Main flow	1. User selects a channel. 2. User types a message and presses send. 3. System checks permission and validity. 4. System saves the message. 5. System delivers it to channel members.
Alternate flow	If the user has no permission or the message is invalid, the system shows an error and does not send it.
Postcondition	Message is saved and visible in the channel.

Chapter 13: Data Flow Diagram (DFD)

A DFD shows how information moves through a system. Rectangles are external entities, circles are processes, open boxes are data stores, and arrows are data flows.

13.1 Level 0 (Context Diagram)

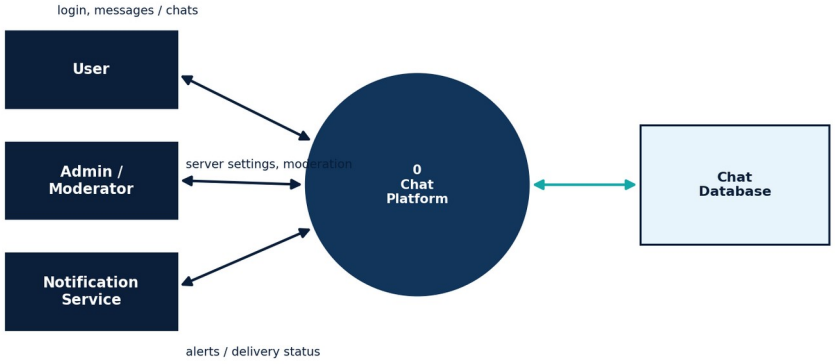


Figure 13.1: Level 0 DFD

13.2 Level 1 DFD

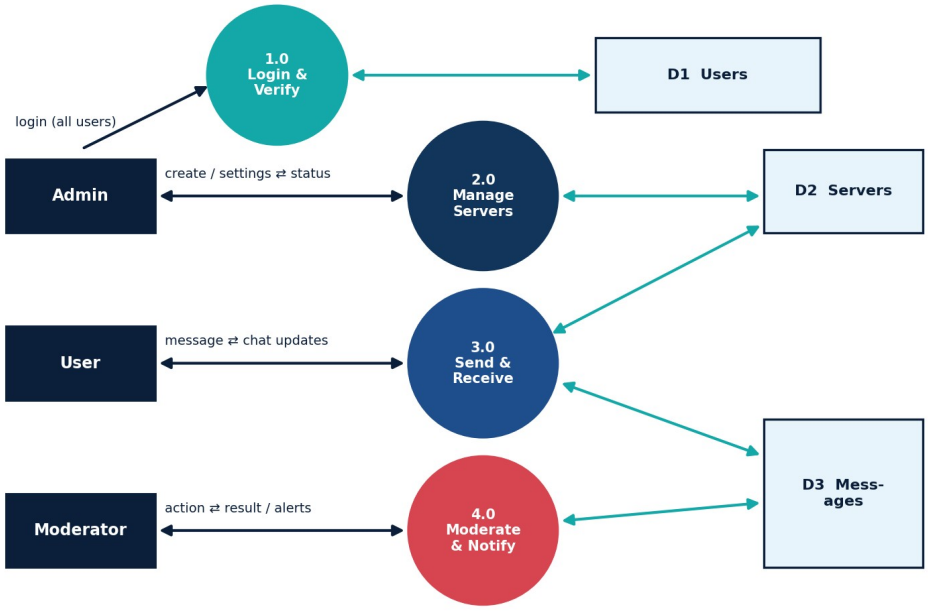


Figure 13.2: Level 1 DFD

Explanation: All users log in through process 1.0, which checks the Users store (D1). The admin's requests go to process 2.0 (Manage Servers), which reads and updates the Servers store (D2). Messages from users go to process 3.0 (Send & Receive), which checks permissions in D2 and saves messages in the Messages store (D3). Moderator actions go to process 4.0 (Moderate & Notify), which updates D3 and sends alerts.

Chapter 14: Database Design

The Entity-Relationship (ER) diagram shows the main tables and how they are related. Many users can join many servers, a server has many channels, a channel contains many messages, and a user sends many messages.

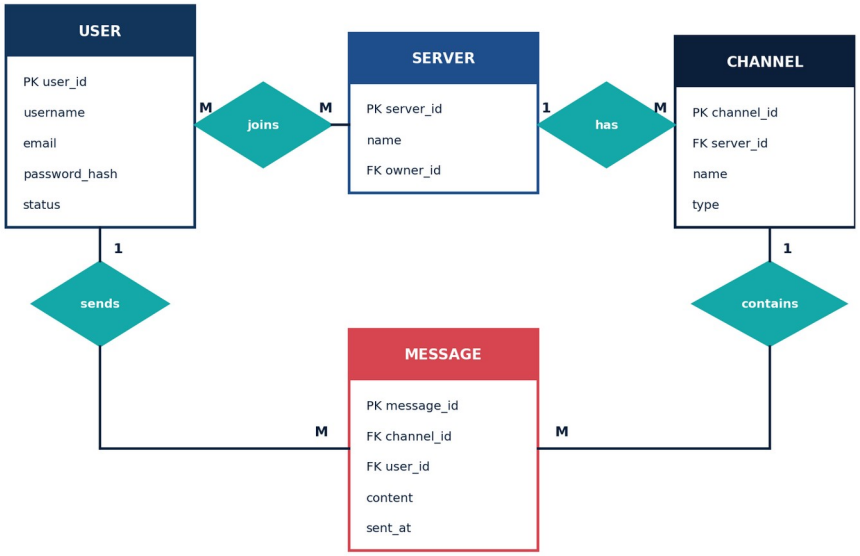


Figure 14.1: Entity-Relationship (ER) diagram

Table 14.1: Table structure

Table	Fields
USER	user_id (PK), username, email, password_hash, status
SERVER	server_id (PK), name, owner_id (FK)
CHANNEL	channel_id (PK), server_id (FK), name, type (text / voice)
MESSAGE	message_id (PK), channel_id (FK), user_id (FK), content, sent_at
MEMBERSHIP	user_id (FK), server_id (FK), role (links users and servers)

PK means primary key (a unique ID) and FK means foreign key (a link to another table). The MEMBERSHIP table resolves the many-to-many relationship between users and servers. Passwords are stored only as hashes.

Chapter 15: UML Design Models

Besides the use case diagram, three more UML diagrams explain the design: the class diagram (structure), the sequence diagram (order of messages) and the state diagram (behaviour over time).

15.1 Class Diagram

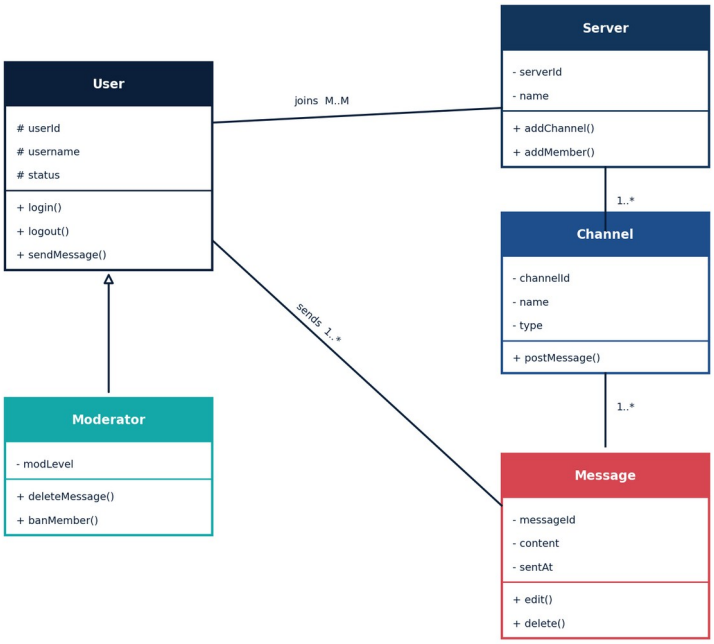


Figure 15.1: UML class diagram of the main classes

15.2 Sequence Diagram

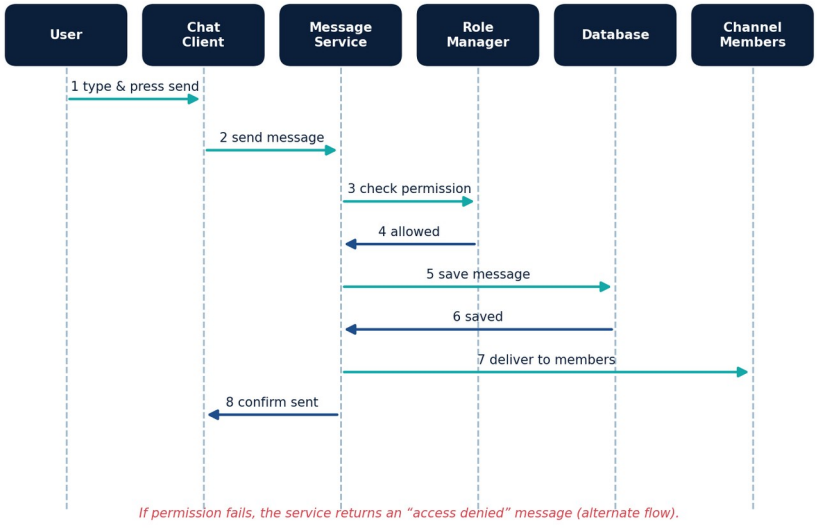


Figure 15.2: UML sequence diagram for sending a message

15.3 State Diagram

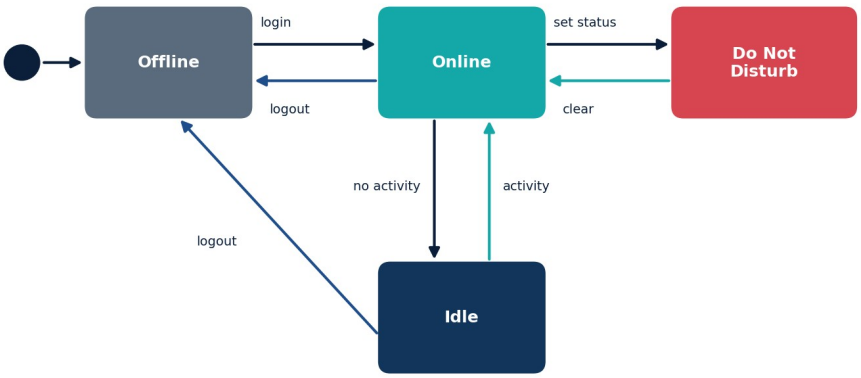


Figure 15.3: UML state diagram of user presence

Explanation: A user starts Offline. After login the user becomes Online. With no activity the user becomes Idle and returns to Online on activity. A user can set Do Not Disturb and clear it later. Logging out moves the user to Offline.

Chapter 16: Software Development Life Cycle (SDLC)

SDLC is the step-by-step process of building software.



Figure 16.1: SDLC phases

Table 16.1: SDLC phases and activities

Phase	Activity for this case study
Planning	Define goals, scope, users and feasibility.
Requirements	Collect needs of members, admins and moderators.
Design	Prepare architecture, database, flowchart, UML and DFD.
Implementation	Code the services, database and client screens.
Testing	Unit, integration, system and acceptance testing.
Deployment	Release the platform to servers and client apps.
Maintenance	Fix bugs, improve performance and add features.

For a platform like this, an iterative approach works well: a first version (login, servers, text channels) is released and tested, and feedback is used to add roles, voice and moderation in later versions.

Chapter 17: Implementation Overview

This chapter gives a simple idea of how the design can be coded. It is a teaching sketch only.

17.1 Suggested Technology

- Client: HTML, CSS and JavaScript (web), or a desktop / mobile app.
- Server: Node.js, Python or Java providing an API.
- Real-time connection: WebSocket (IETF RFC 6455).
- Database: MySQL or PostgreSQL.
- Each module in the architecture becomes a separate service or class.

17.2 Send Message Process (Pseudocode)

```
FUNCTION send_message(user_id, channel_id, text):
  IF NOT is_logged_in(user_id):
      RETURN "Please log in"

  channel = FIND_CHANNEL(channel_id)
  IF channel IS NULL:
      RETURN "Channel not found"

  IF NOT HAS_PERMISSION(user_id, channel, "send"):
      RETURN "Access denied"

  IF text IS EMPTY OR LENGTH(text) > max_length:
      RETURN "Invalid message"

  message = SAVE_MESSAGE(channel_id, user_id, text, NOW)
  FOR each member IN ONLINE_MEMBERS(channel):
      PUSH(member, message)
  RETURN "Sent"
```

17.3 Permission Check (Pseudocode)

```
FUNCTION has_permission(user_id, channel, action):
  role = GET_ROLE(user_id, channel.server_id)
  IF role IS NULL:
      RETURN FALSE            // not a member
  IF user_id IS MUTED:
      RETURN FALSE
  RETURN action IN role.permissions
```

Note: max_length and the permission lists are set by the platform's rules; they are not fixed by this project.

17.4 Coding Practices

- Keep functions small and test each one alone.
- Validate every input before using it.
- Never trust data sent from the client; check permissions on the server.
- Use meaningful names and comments for easy maintenance.

Chapter 18: Security and Risk Management

A chat platform stores accounts and private conversations, and many people use it, so security and safety are very important. Risk management means identifying, assessing, reducing and monitoring risks.

Table 18.1: Security measures

Area	Measures
Authentication	Login with password; lock or slow down after repeated failures.
Password protection	Store passwords only as strong hashes, never as plain text.
Authorisation	Role-based permissions checked on the server for every action.
Secure connection	Use encrypted connections (HTTPS / TLS) for data in transit.
Input validation	Clean all inputs to block script injection and database attacks.
Rate limiting	Limit how fast a user can send messages to reduce spam.
Moderation and reporting	Delete messages, mute or ban members, and allow users to report abuse.
Audit log	Record important admin and moderator actions.

Table 18.2: Sample risk table

Risk	Possible Effect	Control
Account takeover	Misuse of user’s account	Strong passwords, lockout, secure login
Spam and abuse	Poor user experience	Rate limiting and moderation
Harmful content	Users may be harmed	Reporting, deletion and bans
Data leak	Private data exposed	Access control and encryption
Server overload	Service becomes slow or down	Scalable design and monitoring
Script injection	Attack through messages	Input validation and output encoding

Chapter 19: Software Testing

Testing checks that the software does what it should. The test cases below are samples only.

Table 19.1: Sample test cases

ID	Test Case	Input / Condition	Expected Result
TC01	Register user	Valid new details	Account created
TC02	Invalid login	Wrong password	Access denied; error message
TC03	Valid login	Correct ID and password	Access granted; user Online
TC04	Join server	Valid invite	User added to the server
TC05	Send message	Member with permission	Message saved and delivered
TC06	No permission	Member without send permission	Refused; access denied shown
TC07	Empty message	No text entered	Message rejected
TC08	Delete message	Moderator deletes a message	Message removed from channel
TC09	Mute member	Moderator mutes a member	Member cannot send messages
TC10	Presence change	No activity for set time	Status becomes Idle
TC11	Mention notification	User mentioned in channel	Notification sent to that user
TC12	Create channel	Admin creates text channel	Channel appears in the server
TC13	Edit profile	Change display name	Name updated

19.1 Levels of Testing

- Unit testing: each module is tested alone.
- Integration testing: modules are tested together.
- System testing: the complete platform is tested.
- Acceptance testing: users check that their needs are met.

19.2 Testing Methods

- Black-box testing: test inputs and outputs without looking at the code.
- White-box testing: test internal paths and decisions in the code.
- Boundary value testing: test at limits such as message length.
- Load testing: check behaviour when many users are active.

Chapter 20: Ethical and Professional Considerations

Software engineers who build communication platforms have a responsibility to protect users. The ACM/IEEE Software Engineering Code of Ethics asks engineers to act in the public interest.

- Privacy: protect private messages and personal data; collect only what is needed.
- Safety: provide tools to report and handle harassment and harmful content.
- Fairness: apply moderation rules clearly and equally.
- Honesty: be clear about what data is stored and how it is used.

- Accessibility: design screens that more people can use.
- Accountability: keep logs so that decisions can be reviewed.

This project follows these ideas by using no real user data and by including permissions, moderation and audit features in the design.

Chapter 21: Advantages

A Discord-style chat and communication platform offers the following advantages:

- All communication is in one place.
- Real-time delivery of messages.
- Separate channels keep topics organised.
- Roles and permissions give control over who can do what.
- Built-in moderation tools help to keep communities safe.
- Text and voice options suit different needs.
- Message history makes it easy to find older information.
- Members can join from different devices.
- Notifications keep users informed.
- Communities can grow without changing the basic design.
- Modular design makes testing and updates easier.
- Models such as UML, DFD and ER diagrams make the system easy to understand.
- The case study helps students learn the real steps of the SDLC.

Chapter 22: Limitations

This case study and the proposed design also have some limitations:

- The platform needs a stable internet connection.
- The complete software is not built in this case study.
- Real-time systems are complex and costly to build and run.
- Spam, harassment and misuse are hard to control completely.
- Large servers need strong infrastructure to stay fast.
- Security needs constant attention; weak passwords can cause risks.
- Private data must be protected, or privacy can be harmed.
- Moderation depends partly on human moderators.
- Heavy use can distract users from other work.
- Only a conceptual design and sample test cases are presented.

- Testing cannot prove that every possible fault has been found.
- Software updates must be controlled carefully, because new changes can introduce errors.
- Voice and video features are covered only at a high level.
- The design does not describe the real internal systems of any company.

Chapter 23: Future Scope

- Better automatic detection of spam and harmful content, with human review.
- Video calls and screen sharing.
- Bots and integrations with other tools.
- Stronger privacy controls and optional end-to-end encryption.
- Better search across messages and files.
- Accessibility improvements such as screen-reader support.
- A working prototype built in a classroom using WebSocket.
- Analytics dashboards for server admins.

Chapter 24: Conclusion

This case study showed how Software Engineering concepts apply to a Discord-style chat and communication platform. Requirements, a layered architecture, a flowchart, UML, DFD and ER models, the SDLC, security measures and test cases together help to build dependable software. Real-time delivery, permissions and moderation are the key design points.

The study also showed why careful testing, security and ethics matter when software connects many people. This is an independent educational project; the design is conceptual and uses no real user data.

The work has improved my understanding of the Software Engineering subject and how its models are used in real problems.

References

[1] I. Sommerville, Software Engineering, Pearson Education.

[2] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner’s Approach, McGraw-Hill.

[3] P. Jalote, An Integrated Approach to Software Engineering, Springer.

[4] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, Pearson.

[5] A. S. Tanenbaum and D. J. Wetherall, Computer Networks, Pearson.

[6] Object Management Group (OMG), Unified Modeling Language (UML) Specification, omg.org.

[7] I. Fette and A. Melnikov, The WebSocket Protocol, IETF RFC 6455, rfc-editor.org.

[8] ACM/IEEE-CS, Software Engineering Code of Ethics and Professional Practice, acm.org.

[9] OWASP Foundation, OWASP Top 10 Web Application Security Risks, owasp.org.

[10] Discord Developer Documentation, discord.com/developers/docs (for general understanding of platform concepts).

Appendix A: Requirements Traceability Matrix

Traceability links each requirement to a module and a test case, so nothing is left untested.

Table A.1: Requirement traceability

Requirement	Module(s)	Test Case(s)
FR1 Register / login	Authentication Service	TC01, TC02, TC03
FR2 Create / join servers	Channel & Role Manager	TC04
FR3 Create channels	Channel & Role Manager	TC12
FR4 Send / receive messages	Message Service	TC05, TC07
FR5 Roles and permissions	Channel & Role Manager	TC06
FR6 Moderation	Moderation Module	TC08, TC09
FR7 Presence / notifications	Presence & Notification Service	TC10, TC11
FR8 Edit profile	Authentication Service	TC13